



Chapter S1H: Binary Search Tree

CONTENT

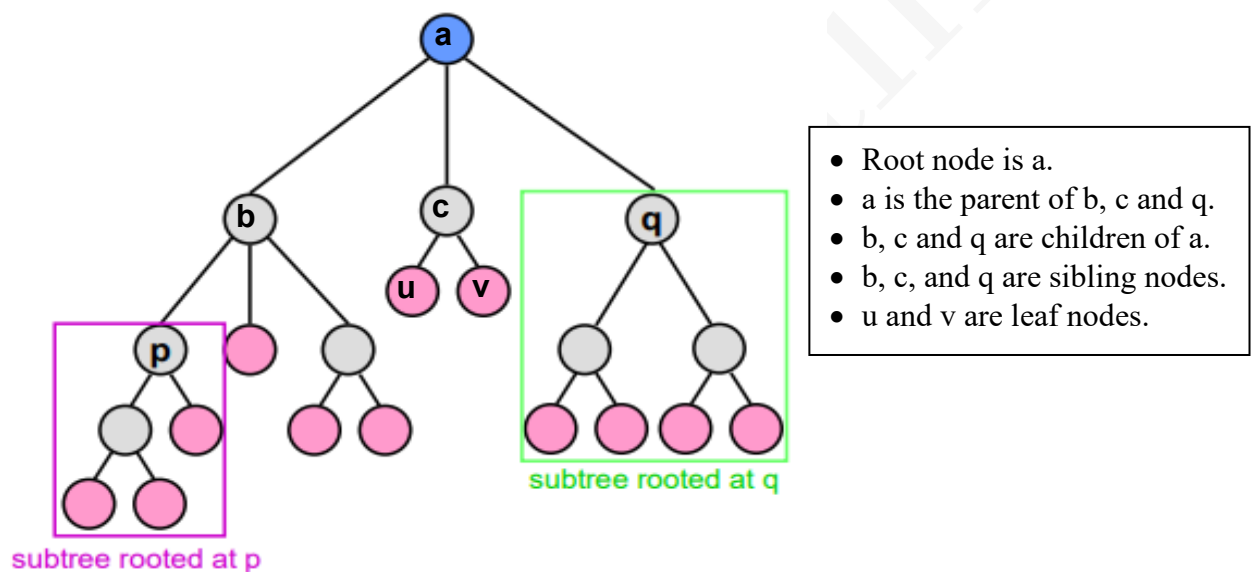
No.	Topic	Syllabus Guide
1	Tree	
2	Binary Tree	1.3.6
3	Binary Search Tree 3.1 Searching for a node 3.2 Inserting a node 3.3 Deleting a node	1.3.6
4	Tree Traversal 4.1 Preorder traversal 4.2 Inorder traversal 4.3 Postorder traversal	1.3.7
5	Array Implementation of BST	2.3.3
6	Linked List Implementation of BST 6.1 Pseudocode to initialise a BST 6.2 Pseudocode for find operation 6.3 Pseudocode for insert operation	2.3.3
7	Tutorial	
8	Programming Exercise	

1. TREE

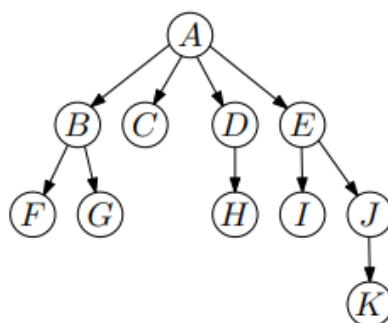
A tree is an **abstract data type (ADT)** that **stores elements hierarchically as a non-linear data structure**. With the exception of the top element, **each element** in a tree **has a parent element** and **zero or more children elements**. It organises its data hierarchically as a collection of **elements/entities**, known as **nodes** that are connected by **edges/branches**.

Basic Terminologies

- Root node: Topmost node in the tree.
- Parent node: Every node (except the root) in a tree is connected by a directed edge/branch from exactly 1 other node. This node is called a parent.
- Child node: A node directly connected to another node when moving away from the root.
- Leaf node: Node with no child
- Subtree: the subtree rooted at x can be defined as the tree of all descendants of x that has node x as root.

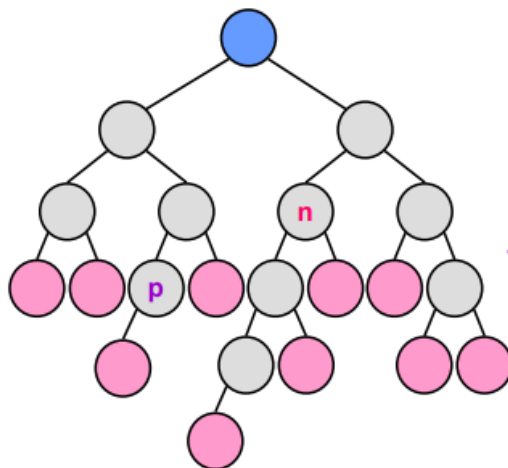


- A path is defined as a sequence of nodes along the edges of a tree.
- The **depth** of a node n is the **number of edges** from **n** to the tree's **root node**.
- The **height** of the tree is the **number of edges** on the **longest path** from the **root** to **leaf**.
- The **height** of a node n is the number of edges on the **longest path** from **n** to a **leaf**.
- A **leaf node** will have a **height** of 0.
- A **root node** will have a **depth** of 0.



Node	Depth	Height
A	0	3
B	1	1
C	1	0
D	1	1
E	1	2
F	2	0
G	2	0
H	2	0
I	2	0
J	2	1
K	3	0

Quiz: Fill in the height and depths

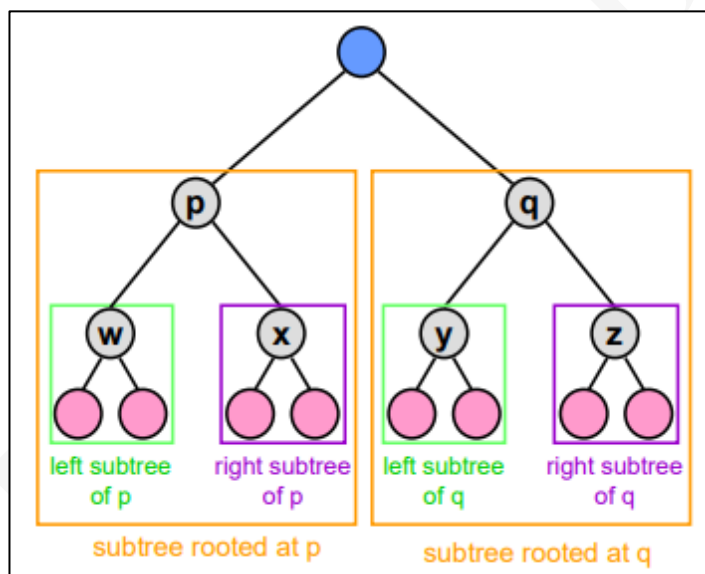


- Height of the tree =
- Depth of node n =
- Depth of node p =

2. BINARY TREE

Binary Tree is a tree data structure which has the following properties:

1. Every node has at most 2 subtrees/children nodes (one or both can be non-existent).
2. Each child node is labelled as being either a '**left-child**', or a '**right-child**'.
3. A left child precedes a right child in the order of children of a node.



Subtree:

- the left subtree of a node is the tree rooted at the left child of the node.
- the right subtree of a node is the tree rooted at the right child of the node.

Common operations

- Create an empty tree.
- Test whether a tree is empty (isEmpty).
- Insert a new node.
- Delete a node.
- Search for a node.
- List (traverse) the nodes of the tree.

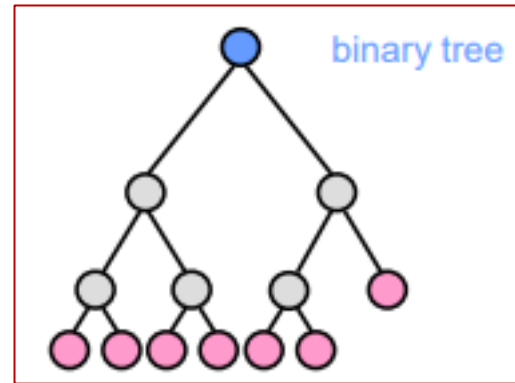
In a binary tree of height h ,
The **maximum** number of nodes is:

$$2^{h+1} - 1$$

The **maximum** number of **leaf** nodes is:

$$2^h$$

Quiz: Can you figure out why this is so?



3. BINARY SEARCH TREE (BST)

A **binary search tree (BST)** is a **binary tree** where each node contains a value from a **well-ordered set**.

For each node n in a BST the following properties hold:

- Every node in the **left subtree of n** contains a value which is **smaller** than the value in the node n .
- Every node in the **right subtree of n** contains a value which is **larger** than the value in the node n .

In a BST, all keys (*value of a node*) in **left subtree of a key must be smaller** and all keys in **right subtree must be greater**.

By definition, a BST has distinct keys and duplicates in binary search tree are not allowed. However, there is nothing that prevents duplicate keys from being implemented in a BST. It just means that the implementation will be more complex to manage the duplicate keys.

	Consider a binary tree with 9 nodes. The root of the tree contains the value 8. The leaf values are 1, 4, 7 and 13. Each node in the left subtree has a value smaller than 8, while each node in the right subtree has a value larger than 8. Therefore, this is a binary search tree, since the corresponding property holds for each node in the tree.
	In this binary tree, there exists a node of key “2” that is in the right subtree of key “3”. Therefore, this is <u>NOT</u> a binary search tree.

<pre> graph TD 10((10)) --> 3((3)) 10 --> 15((15)) 3 --> 1((1)) 3 --> 11((11)) 11 --> 4((4)) 11 --> 12((12)) </pre>	In this binary tree, there exist 2 nodes of keys “11” and “12” that is in the left subtree of key “10”. Therefore, this is <u>NOT</u> a binary search tree.
---	--

3.1 Searching for a node in a BST

- To search for a node, start at the root. Compare search value with value at root node. If **search value is same as value at root node**, then **node is found**.
- If **search value is less than value at root node**, then access left subtree, otherwise access right subtree.
- If **root of subtree is not null**, then compare search value with value at root of subtree and traverse left or right subtree accordingly.
- Repeat the comparisons and traversing of nodes **until value is found** or subtree is null, where **value is not found**.

<pre> graph TD 8((8)) --> 3((3)) 8 --> 10((10)) 3 --> 1((1)) 3 --> 6((6)) 6 --> 4((4)) 6 --> 7((7)) 10 --> 14((14)) 14 --> 13((13)) </pre>	Find node with value 8 1. Traversal will start from root node with value 8. 2. 8 (found). Find node with value 7 1. Traversal will start from root node with value 8. 2. 8 → 3 → 6 → 7 (found). Find node with value 14 1. Traversal will start from root node with value 8. 2. 8 → 10 → 14 (found). Find node with value 5 1. Traversal will start from root node with value 8. 2. 8 → 3 → 6 → 4 → null (not found).
--	---

3.2 Inserting a node into a BST

- To insert a new node into the tree, start at the root. If tree is null, then insert new node as root node with new value and exit.
- If the new value is less than the value at the node, move left, otherwise move right.
- If root of subtree is null, then insert new node at root and exit.
- Otherwise, repeat comparing new value with value at node and traverse either left or right, until there are no further nodes to compare.
- The new node is inserted at this point by entering the data and **setting up the link from the previous node**.

Quiz: The following items are inserted into an **empty** Binary Search Tree. **Draw** the tree diagram **after** every node is successfully inserted into a BST.

{50, 80, 30, 20, 100, 75, 25, 15}

3.3 Deleting a node from a BST

To delete a node from a BST, first locate the node to be deleted. Once the node to be deleted has been found, there are **three** different cases to consider:

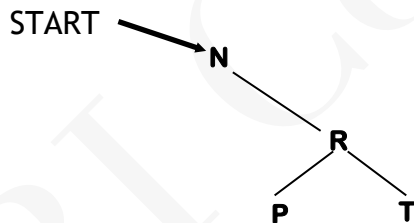
1. If the node to be deleted has **no** children.
2. If the node to be deleted has only **one** child.
3. If the node to be deleted has **two** children.

Case 1: If node has no children

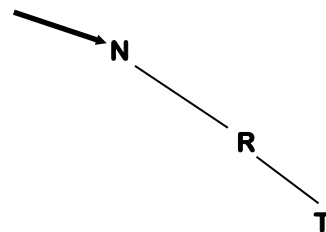
If node has no children,

- Remove required node.
- Pointer pointing to required node is set to null.

BEFORE DELETION
(Node to delete: P)



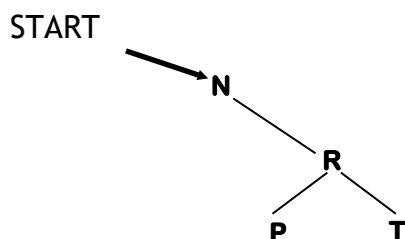
AFTER DELETION



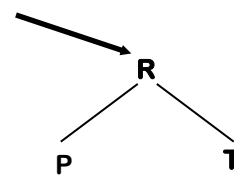
Case 2: If node has only one child

Whether the child is to the left or right, or the child has children of its own does not matter. **The child node simply takes the place of the node being removed.** The deleted node's parent's left or right pointer (as appropriate) is updated to point to its grandchild.

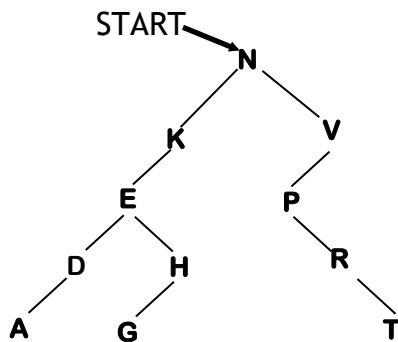
BEFORE DELETION
(Node to delete: N)



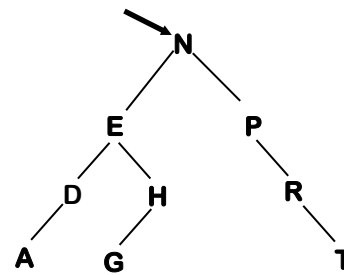
AFTER DELETION



BEFORE DELETION
(Nodes to delete: K,V)



AFTER DELETION

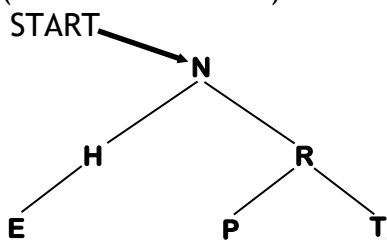


Case 3: If node has two children

Replace node with the *largest node in its left subtree*, and then delete node.

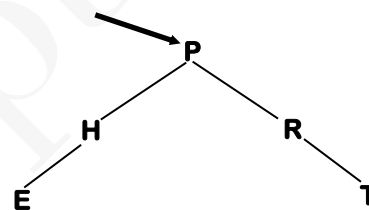
Alternatively, node can also be replaced with the *smallest node in its right subtree*, and then delete node.

BEFORE DELETION
(Node to delete: N)



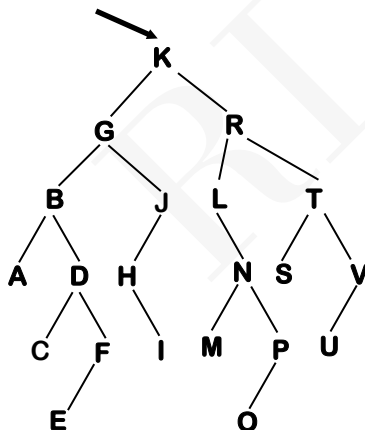
(P is smallest node in right subtree of N)

AFTER DELETION

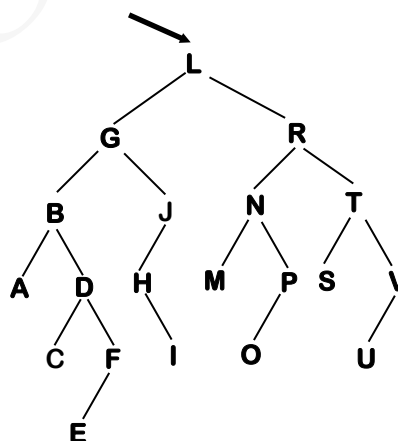


(Nodes to delete: K then R)

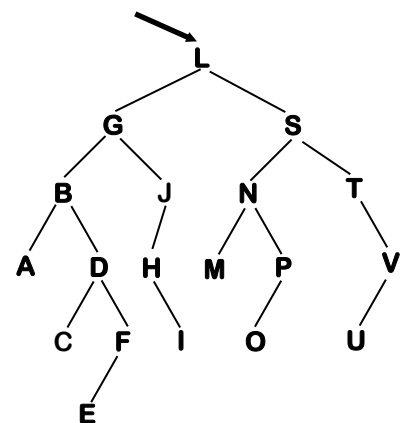
START



(L is smallest node in right subtree of K)



(S is smallest node in right subtree of R)

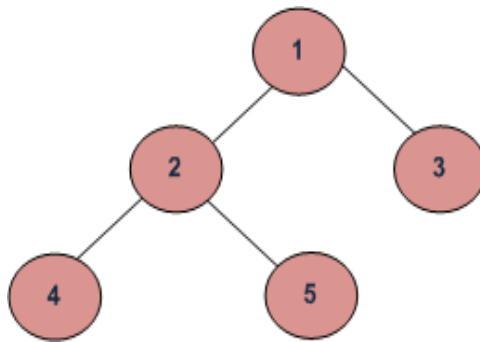


4. TREE TRAVERSAL

Traversing a tree T is a **process to visit all the nodes** of a tree that are connected via edges/branches. The first node to be visited in a non-empty tree will always **start from the root node**.

It can be done in the following ways:

1. **Breadth first** traversal: visits node by level from the root.
2. **Depth-first** traversal: visits nodes as far ahead as possible before backing up. There are 3 types of depth-first traversal:
 - a. **Preorder** traversal: a node is processed **before** its children.
 - b. **Inorder** traversal: a node is processed **between** its left child and its right child.
 - c. **Postorder** traversal: a node is processed **after** its children.



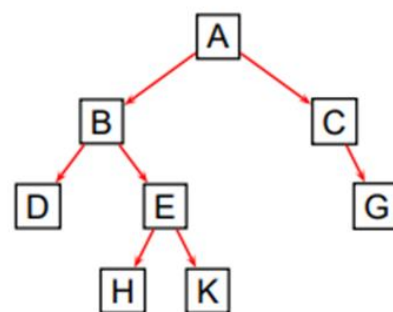
Breadth First or Level Order Traversal: 1 2 3 4 5

Depth First Traversals:

- | | |
|---|-----------|
| (a) Preorder (Root, Left, Right): | 1 2 4 5 3 |
| (a) Inorder (Left, Root, Right): | 4 2 5 1 3 |
| (c) Postorder (Left, Right, Root): | 4 5 2 3 1 |

4.1 Preorder Traversal

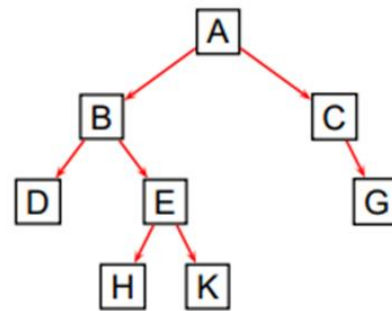
1. Access the data of the root node.
2. Traverse the left subtree by recursively calling the preorder function.
3. Traverse the right subtree by recursively calling the preorder function.



Order of access:
A B D E H K C G

4.2 Inorder Traversal

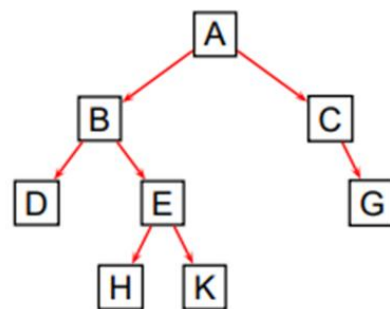
1. Traverse the left subtree by recursively calling the inorder function.
2. Access the data of the root node.
3. Traverse the right subtree by recursively calling the inorder function.



Order of access:
D B H E K A C G

4.3 Postorder Traversal

1. Traverse the left subtree by recursively calling the postorder function.
2. Traverse the right subtree by recursively calling the postorder function.
3. Access the data of the root node



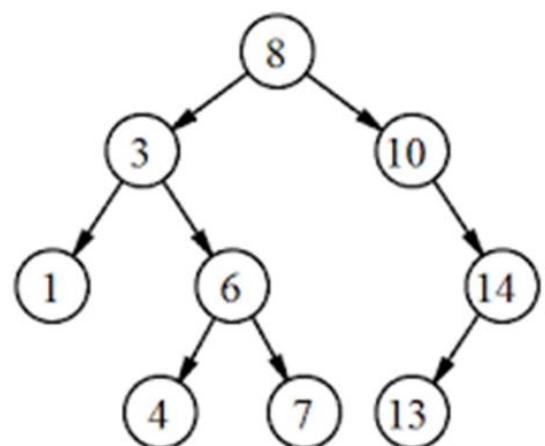
Order of access:
D H K E B G C A

Quiz: Perform the tree traversal of the BST.

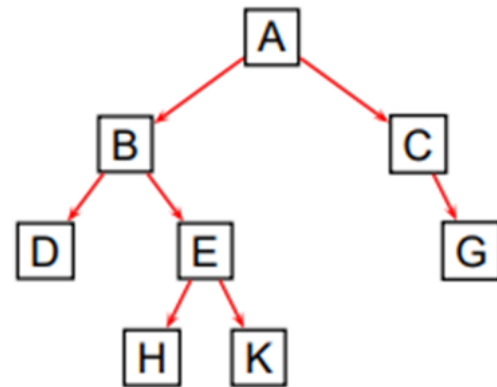
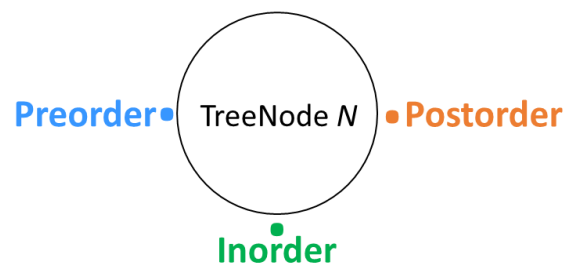
Preorder:

Inorder:

Postorder:



A “quick” way to do tree traversal



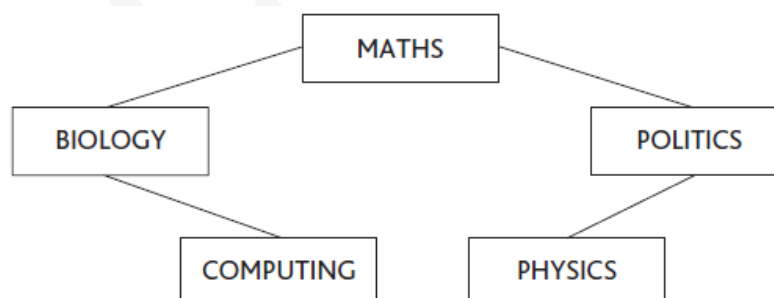
5. ARRAY IMPLEMENTATION OF BST

A BST may be implemented using arrays. Here, we use three 1-dimensional arrays.

- A fixed sized array of STRINGS to hold the data.
- A fixed sized array of INTEGERS to track left pointers.
- A fixed sized array of INTEGERS to track right pointers.

The “**Root**” pointer stores the index position of the root item in the tree.

The tree below contains data for subjects offered at a school.

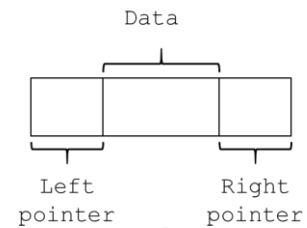


Index		Data	LeftLink	RightLink
1		MATHS	3	2
2		POLITICS	4	NULL
3		BIOLOGY	NULL	5
4		PHYSICS	NULL	NULL
5		COMPUTING	NULL	NULL

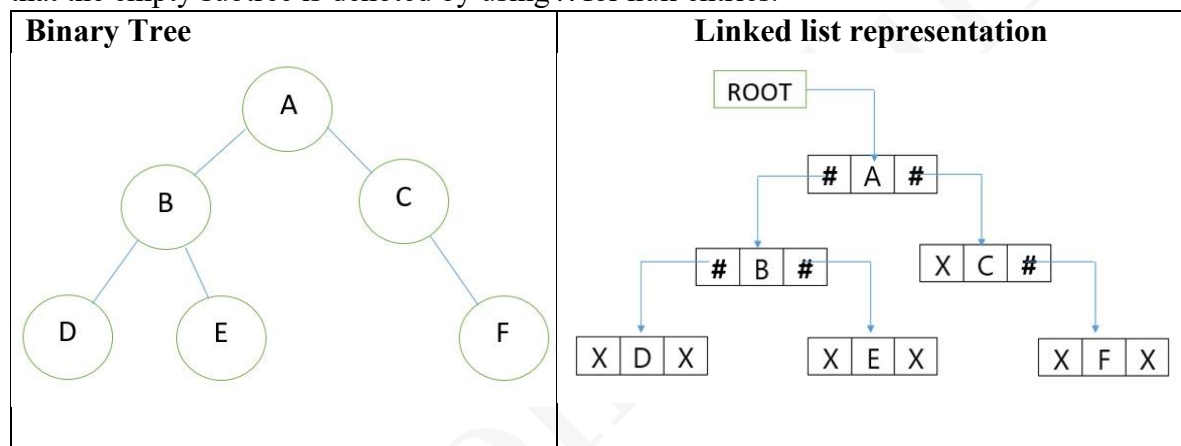
6. LINKED LIST IMPLEMENTATION OF BST

A BST may be implemented as a linked list. In a linked list, the nodes are linked to each other through the parent-child relationship associated with trees.

A node is shown on the right. Left and right pointers are two link fields to store the **address of left child and right child of a node** respectively. With this representation, **it is not possible to directly access any nodes in the BST except for the root node. However, a node can still be accessed by traversing from the root node of the BST.**



Consider a binary tree T in the figure. A schematic diagram of the linked list representation of T appears in the following figure. Observe that each node is drawn with three fields, and that the empty subtree is denoted by using X for null entries.



Node content			
Addr	...	...	...
x+8	null	M	x+6
x+7	null		null
x+6	null	U	null
x+5	x+2	G	x+8
x+4	x+7		null
x+3	null	A	null
x+2	x+3	D	x+1
x+1	null	E	null
	...	...	...

Binary Search Tree

Root pointer: x+5

Free pointer: x+4

Draw the linked list representation of the BST represented by the figure on the left.

6.1 Pseudocode to Initialise a BST using Array of Tree Nodes

```
// NullPointer should be set to -1 if using 0-based indexing
// Declare record type to store data and pointers
// This implementation uses 1-based indexing
// for an array size of 7
// when BST is empty: RootPointer = 0, FreePtr = 1

CONSTANT NullPointer = 0

TYPE TreeNode
    DECLARE Data : STRING
    DECLARE LeftPointer : INTEGER
    DECLARE RightPointer : INTEGER
END TYPE

DECLARE RootPointer : INTEGER
DECLARE FreePtr : INTEGER
DECLARE Tree[1:7] OF TreeNode

PROCEDURE InitialiseTree
    RootPointer ← NullPointer //set start pointer to null
    FreePtr ← 1 //set first free node
    Index ← 1
    WHILE Index < 7 // link all nodes to make a free list
        Tree[Index].LeftPointer ← Index + 1
        Index ← Index + 1
    ENDWHILE
    Tree[7].LeftPointer ← NullPointer
END PROCEDURE
```

Quiz: Illustrate the Binary Search Tree based on the pseudocode above

7	
6	
5	
4	
3	
2	
1	
...	

Binary Search Tree

RootPointer:

FreePtr:

6.2 Pseudocode for Search Operation

```

FUNCTION FindNode(Searchitem) RETURNS INTEGER
    current ← RootPointer

    // while pointer not null and search item not found
    WHILE current <> NullPointer AND Tree[current].Data <> Searchitem
        IF Searchitem < Tree[current].Data THEN
            current ← Tree[current].LeftPointer    // access left
        ELSE
            current ← Tree[current].RightPointer  // access right
        ENDIF
    ENDWHILE

    RETURN current //NullPointer (ie.0) returned if not found
END FUNCTION

```

Quiz: Complete the trace table for **FindNode('A')**

	...	...	...
x+8	null	M	x+6
x+7	null		null
x+6	null	T	null
x+5	x+2	G	x+8
x+4	x+7		null
x+3	null	A	null
x+2	x+3	D	x+1
x+1	null	E	null
	...	...	...

Binary Search Tree

RootPointer: x+5

FreePtr: x+4

T

current	Tree[current].Data	RETURN

Similarly, complete the trace table for **FindNode('F')**

current	Tree[current].Data	RETURN

6.3 Pseudocode for Insert Operation

```

PROCEDURE InsertNode(Newitem)
    IF FreePtr <> NullPointer THEN                // free node is available
        // prepare node for insertion
        NewNodePtr ← FreePtr                    // set the new node index
        FreePtr ← Tree[FreePtr].LeftPointer    // update the free node index
        Tree[NewNodePtr].Data ← Newitem        // update data of new node
        Tree[NewNodePtr].LeftPointer ← NullPointer
        Tree[NewNodePtr].RightPointer ← NullPointer

        //case 1: BST is empty, insert new node as the root
        IF RootPointer = NullPointer THEN
            RootPointer ← NewNodePtr            // insert new node at root

        //case 2: BST is not empty. Traverse from root to insert new node as leaf
        ELSE
            current ← RootPointer                // start at the root of the tree
            WHILE current <> NullPointer          // while not a leaf node
                previous ← current                // remember the current node
                IF Newitem < Tree[current].Data THEN // follow left pointer
                    TurnedLeft ← TRUE
                    current ← Tree[current].LeftPointer
                ELSE                             // follow right pointer
                    TurnedLeft ← FALSE
                    current ← Tree[current].RightPointer
                ENDIF
            ENDWHILE
            // update parent's pointer to point to new node
            IF TurnedLeft = TRUE THEN
                Tree[previous].LeftPointer ← NewNodePtr
            ELSE
                Tree[previous].RightPointer ← NewNodePtr
            ENDIF
        ENDIF
    ELSE // free node is not available
        OUTPUT "No free node is available"
    ENDIF
ENDPROCEDURE

```

Quiz: The following insert operations are to be performed on a BST in the order given below. Given that steps 1 and 2 have already been performed, trace `InsertNode("Doris")` using the table below:

1. `InsertNode("Helen")`
2. `InsertNode("Gina")`
3. `InsertNode("Kelly")`
4. `InsertNode("Cindy")`
5. `InsertNode("Molly")`
6. `InsertNode("Doris")`

7	0		0
6	7		0
5	6		0
4	5		0
3	4		0
2	0	Gina	0
1	2	Helen	0
	...	...	...

Tree

RootPointer:

1

FreePtr:

3

FreePtr	NewNodePtr	current	Tree[current].Data	TurnedLeft	previous

7. TUTORIAL

1. Given: START - Start pointer pointing to root of tree.
 NODE[A,1] - Left pointer of Ath node.
 NODE[A,2] - Data of Ath node.
 NODE[A,3] - Right pointer of Ath node.

START	1	A	NODE[A,1]	NODE[A,2]	NODE[A,3]
		1	2	56	3
		2	4	32	0
		3	5	85	6
		4	7	20	8
		5	9	72	0
		6	0	95	0
		7	0	15	0
		8	0	22	0
		9	0	62	10
		10	0	68	0

where 0 is null pointer

- Construct the above binary tree.
 - Write down the order of the elements of the tree when it is traversed:
 - Preorder
 - Inorder
 - Postorder
2. The following data items are to be entered into a binary search tree in the order given:
 London, Paris, Rome, Berlin, Lisbon, Madrid, Canberra, Vienna, Amsterdam
- Draw a diagram to show how these values will be stored.
 - Describe how this binary search tree can be implemented using arrays.
 - If Madrid is being searched for in this binary tree, list the data items which have to be accessed.
 - Write an algorithm that will print the data items from the binary search tree in ascending order.
3. [ACJC/2008/Prelim/9754/P1/Q8] A binary tree holds a quantity of names, in such a way that for any node in the tree, all nodes in the subtree on the left contain names which are earlier in the alphabet, and all nodes in the subtree to the right contain names which are later in the alphabet. There are no duplicate names.
- Draw the alphabetically ordered binary tree after all the names James, Henry, Julia, Rachel, John, Edward, June, Emma, Jennifer and Stephen have been added in the given order.
 - Write an algorithm for the insertion of an item into an alphabetically ordered binary tree.
 - Write down the output if the tree traversed using
 - Pre-order
 - In-order
 - Post-order

- (d) Redraw the binary tree in (a) after the name Julia is removed.
- (e) A binary tree abstract data type (ADT) is to be implemented by using nodes with appropriate pointers and storage for names.
 - (i) Describe with an aid of diagram how the above binary tree would be implemented using these nodes.
 - (ii) State two operations that would be required for the ADT.

4. [HCI/2009/Prelim/9754/P1/Q7] An Abstract Data Type (ADT) contains of both data type and associated operations. A binary search tree ADT has the following operations defined:

Operations	Description
Create(x)	Creates an empty binary search tree x
Insert(x, item)	Inserts new value, item, into the BST x
Delete(x, item)	Deletes the value, item, from the BST x
Search(x, item)	Returns True if the value, item is in the BST x, otherwise return False
Count(x)	Returns the number of nodes in the BST x
IsEmptyTree(x)	Returns True if the BST is empty
SortedDisplay(x)	Displays values stored in the binary search tree x in sorted order

The binary search tree is implemented by the use of a collection of nodes that have three parts: the item data and two pointers; the left pointer points to the root node of the left subtree; the right pointer points to the root node of the right subtree. In addition, there is a Root pointer which points to the root node of the binary search tree.

- (a) Given the following sequence of values, explain how these values should be inserted into the binary search tree and draw the diagram of the final generated BST.
60, 50, 30, 70, 80, 35, 57, 75, 32, 40
- (b) Write recursive algorithms to implement the
 - (i) Search operation;
 - (ii) SortedDisplay operation
- (c) Draw two possible BST diagrams after the value, 60, is deleted from the tree.

8. PROGRAMMING EXERCISE

Data can often be efficiently stored and processed using a binary tree. An application is to be developed to store and process students' exam results (names and marks) using a binary tree. Marks are integer values between 0 and 100 inclusive. The maximum number of students is 30.

The program will use a user-defined type `Node` for each node defined as follows:

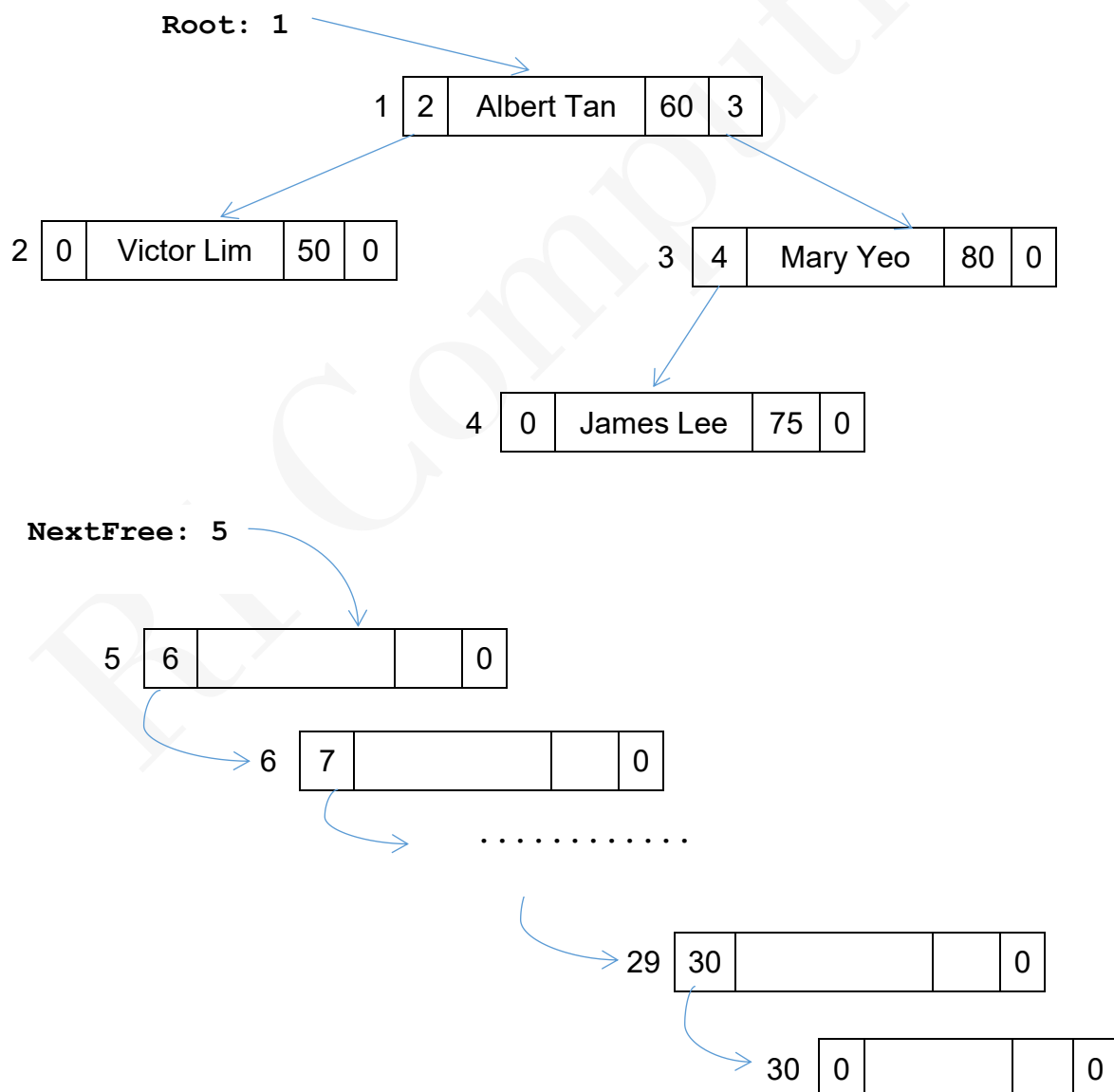
Identifier	Data Type	Description
LeftP	INTEGER	The left pointer for the node
Name	STRING	The node's value for name of student
Mark	INTEGER	The node's value for mark of student
RightP	INTEGER	The right pointer for the node

A linked list is maintained of all the unused nodes which do not form part of the tree. The first available node which is used for a new term is indicated by `NextFree`. Items in the unused list are linked using their left pointers.

The binary tree and linked list are implemented using variables as follows:

Identifier	Data Type	Description
<code>ResultTree</code>	<code>ARRAY[30] : Node</code>	The tree data
<code>Root</code>	<code>INTEGER</code>	Index for the root position of the <code>ResultTree</code> array
<code>NextFree</code>	<code>INTEGER</code>	Index for the next unused node

The diagram shows the binary tree and linked list after four students' results have been added.



1. Write program code to declare all the required variables and create the initial linked list which contains all 30 nodes. Using Object-Oriented Programming techniques, code the properties of the classes as private and access them via class methods. Add statement(s) to initialise the empty tree.

The following pseudocode inserts a data value into the binary search tree structure.

The LastMove variable holds the direction of the previous traversal move as follows:

X – no move yet made
 L – move was to the left
 R – move was to the right

```

PROCEDURE AddData (name, mark) :
  Validate name and mark, exit if error
  IF NextFree = 0
    Output "No free node available"
  ELSE
    Grab node from free list and copy name and mark to it
    IF Root = 0
      Root ← NextFree
    ELSE
      Current ← Root
      LastMove ← 'X'
      REPEAT
        Previous ← Current
        IF mark < mark at ResultTree[Current] THEN
          //move left
          LastMove ← 'L'
          Current ← LeftP of ResultTree[Current]
        ELSE
          //move right
          LastMove ← 'R'
          Current ← RightP of ResultTree[Current]
        ENDIF
      UNTIL Current = 0
      IF LastMove = 'L' THEN
        ResultTree[Previous].LeftP ← NextFree
      ELSE
        ResultTree[Previous].RightP ← NextFree
      ENDIF
    ENDIF
  ENDIF
  NextFree points to next available node
ENDPROCEDURE
  
```

2. Write module **AddData** to add a student's name and mark into the binary search tree.

3. Write a module **DisplayData** to display the value of `Root`, the value of **NextFree** and the contents of **ResultTree** in index order.
4. Write a module **ConstructBST** to construct a binary search tree, get user to input the following data and store them into the binary tree using the **AddData** module.
Input these data in order:

Name	Mark
Ahmad Ibrahim	60
Victor Lim	50
Mary Ho	80
Kannan	75
Joel Tan	85
John Barnes	35
Pan Li	70
Ray Yeo	48

5. Test your program by running **DisplayData** module to display the contents of the binary search tree.
6. Write a module **GetLowest** to determine the lowest mark scored for the exam, with the student's name.
7. Write a module **SearchMoreEqual** to determine all students who **scored at least 70 marks**. Module should display names of students and marks scored.
8. Write a module **ListByScore** that will list all the scores, together with its name, in increasing order.
9. Write a module **CalculateAverageScore** that will calculate and output the average score.